Probabilistic Machine Learning

Block 3 – Unsupervised Learning & Probabilistic Clustering

Andrés Masegosa

Aalborg University

2026

Outline

1. From k-means to Gaussian Mixture Models
2. Closing: What We Covered & What Comes Next

The Same Recipe, One More Time

1. Define
a probabilistic model

→

2. Infer
estimate parameters

→

3. Predict
with uncertainty

	Block 1	**Block 2**	**Block 3**
Problem	Regression	Classification	Clustering
Supervision	Supervised	Supervised	**Unsupervised**
Distribution	Gaussian	Bernoulli	**Mixture**

The Same Recipe, One More Time

1. Define
a probabilistic model

→

2. Infer
estimate parameters

→

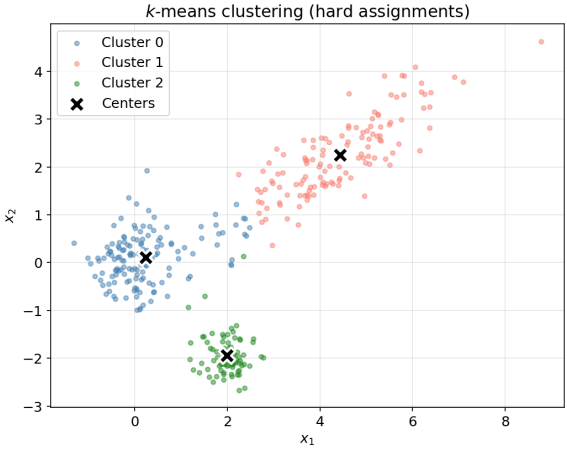
3. Predict
with uncertainty

	Block 1	**Block 2**	**Block 3**
Problem	Regression	Classification	Clustering
Supervision	Supervised	Supervised	**Unsupervised**
Distribution	Gaussian	Bernoulli	**Mixture**

New ingredient: **latent variables** — cluster assignments are not observed.

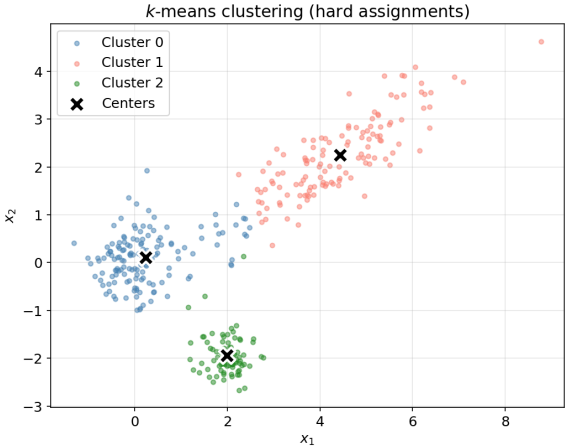
k-means: The Standard Approach

k-**means** assigns each point to the **nearest** cluster center.



k-means: The Standard Approach

k-**means** assigns each point to the **nearest** cluster center.



What If k-means *Were* a Probabilistic Model?

Start from the k-means objective — minimize total squared distance:

$$J_{k\text{-means}} = \sum_{i=1}^{N} \min_{k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

What If k-means *Were* a Probabilistic Model?

Start from the k-means objective — minimize total squared distance:

$$J_{k\text{-means}} = \sum_{i=1}^{N} \min_{k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

Now consider a GMM with three **extreme** constraints:

1. **Equal weights:** $\pi_k = \frac{1}{K}$ for all k
2. **Spherical, identical covariances:** $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$ for all k
3. **Hard assignments:** $\sigma^2 \to 0$

What If k-means *Were* a Probabilistic Model?

Start from the k-means objective — minimize total squared distance:

$$J_{k\text{-means}} = \sum_{i=1}^{N} \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

Now consider a GMM with three **extreme** constraints:

1. **Equal weights:** $\pi_k = \frac{1}{K}$ for all k
2. **Spherical, identical covariances:** $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$ for all k
3. **Hard assignments:** $\sigma^2 \to 0$

Its negative log-likelihood:

$$-\log p(\mathcal{D} \mid \theta) \;=\; -\sum_{i=1}^{N} \log\left(\sum_{k=1}^{K} \frac{1}{K} \cdot \underbrace{\mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \sigma^2 \mathbf{I})}_{\propto \exp\left(-\frac{\|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2}{2\sigma^2}\right)} \right) \xrightarrow{\sigma^2 \to 0} \text{const} + \frac{1}{2\sigma^2} \sum_{i=1}^{N} \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

What If k-means *Were* a Probabilistic Model?

Start from the k-means objective — minimize total squared distance:

$$J_{k\text{-means}} = \sum_{i=1}^{N} \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

Now consider a GMM with three **extreme** constraints:

1. **Equal weights:** $\pi_k = \frac{1}{K}$ for all k
2. **Spherical, identical covariances:** $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$ for all k
3. **Hard assignments:** $\sigma^2 \to 0$

Its negative log-likelihood:

$$-\log p(\mathcal{D} \mid \theta) \;=\; -\sum_{i=1}^{N} \log\left(\sum_{k=1}^{K} \frac{1}{K} \cdot \underbrace{\mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \sigma^2 \mathbf{I})}_{\propto \exp\left(-\frac{\|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2}{2\sigma^2}\right)} \right) \xrightarrow{\sigma^2 \to 0} \text{const} + \frac{1}{2\sigma^2} \sum_{i=1}^{N} \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

Key insight

Minimizing $J_{k\text{-means}}$ = maximizing the likelihood of a GMM with equal, spherical, infinitely-tight clusters. **k-means is a degenerate GMM.**

From k-means to GMM: Relaxing Three Assumptions

k-**means constraint**	**GMM relaxation**	**What changes**
$\pi_k = \frac{1}{K}$ (equal weights)	π_k learned from data	Some clusters more likely
$\mathbf{\Sigma}_k = \sigma^2 \mathbf{I}$ (spherical)	$\mathbf{\Sigma}_k$ full, per-cluster	Clusters can be elongated
Hard: $\sigma^2 \to 0$	Soft: probabilities $\in (0, 1)$	Smooth, uncertain boundaries

From k-means to GMM: Relaxing Three Assumptions

k-means constraint	GMM relaxation	What changes
$\pi_k = \frac{1}{K}$ (equal weights)	π_k learned from data	Some clusters more likely
$\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$ (spherical)	$\boldsymbol{\Sigma}_k$ full, per-cluster	Clusters can be elongated
Hard: $\sigma^2 \to 0$	Soft: probabilities $\in (0, 1)$	Smooth, uncertain boundaries

Assignment equations side by side:

k-means (hard)

$$z_i^* = \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

GMM (soft — "responsibility")

$$z_{ik} = \frac{\pi_k \, \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^{K} \pi_j \, \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

From k-means to GMM: Relaxing Three Assumptions

k-means constraint	GMM relaxation	What changes
$\pi_k = \frac{1}{K}$ (equal weights)	π_k learned from data	Some clusters more likely
$\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$ (spherical)	$\boldsymbol{\Sigma}_k$ full, per-cluster	Clusters can be elongated
Hard: $\sigma^2 \to 0$	Soft: probabilities $\in (0, 1)$	Smooth, uncertain boundaries

Assignment equations side by side:

k-means (hard)

$$z_i^* = \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

GMM (soft — "responsibility")

$$z_{ik} = \frac{\pi_k \, \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^{K} \pi_j \, \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

When $\sigma^2 \to 0$ and all π_k equal, $z_{ik} \to \{0, 1\}$ — the soft assignment **becomes** the hard assignment.

Gaussian Mixture Model: The Probabilistic Version

A GMM defines a **generative story** for each data point:

1. **Choose a cluster:** $z_i \sim \mathrm{Categorical}(\boldsymbol{\pi})$
2. **Generate data from that cluster:** $\mathbf{x}_i \mid z_i = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$

Gaussian Mixture Model: The Probabilistic Version

A GMM defines a **generative story** for each data point:

1. **Choose a cluster:** $z_i \sim \mathrm{Categorical}(\boldsymbol{\pi})$
2. **Generate data from that cluster:** $\mathbf{x}_i \mid z_i = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$

Parameters:

- $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_K)$ — mixture weights (how likely each cluster is)
- $\boldsymbol{\mu}_k$ — cluster centers (means)
- $\boldsymbol{\Sigma}_k$ — cluster shapes (covariances)

Gaussian Mixture Model: The Probabilistic Version

A GMM defines a **generative story** for each data point:

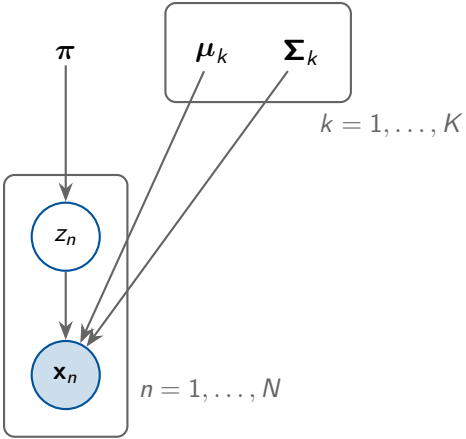
1. **Choose a cluster:** $z_i \sim \mathrm{Categorical}(\boldsymbol{\pi})$
2. **Generate data from that cluster:** $\mathbf{x}_i \mid z_i = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$

Parameters:

- $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_K)$ — mixture weights (how likely each cluster is)
- $\boldsymbol{\mu}_k$ — cluster centers (means)
- $\boldsymbol{\Sigma}_k$ — cluster shapes (covariances)

z_i is a **latent variable**: we observe $\mathbf{x}_i$ but not z_i.

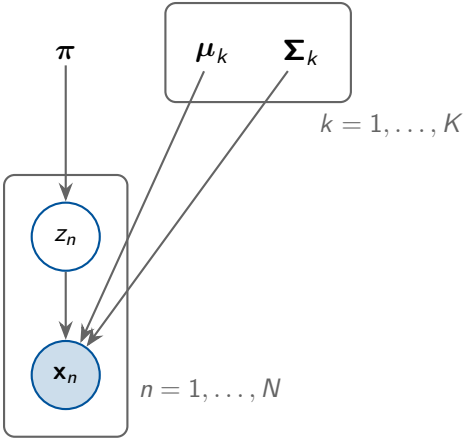
Graphical Model: GMM



Reading the PGM:

- parameter (`pyro.param`)
- observed (`obs=data`)
- latent variable
- plate (`pyro.plate`)
- $\rightarrow$ dependency

Graphical Model: GMM



Reading the PGM:

- parameter (`pyro.param`)
- observed (`obs=data`)
- latent variable
- plate (`pyro.plate`)
- $\rightarrow$ dependency

Key difference with Blocks 1–2: z_n is **not observed** — it is a latent variable (white node) that must be marginalized out.

This is why GMM uses `MixtureSameFamily`: it marginalizes z_n analytically.

k-means vs. GMM

	k-**means**	**GMM**
Assignments	Hard (0 or 1)	**Soft** (probabilities)
Shape	Spherical only	Any (via $\mathbf{\Sigma}_k$)
Uncertainty	None	Per-point uncertainty
Model selection	Heuristic	Likelihood-based
Generative	No	**Yes** — can sample new data

k-means vs. GMM

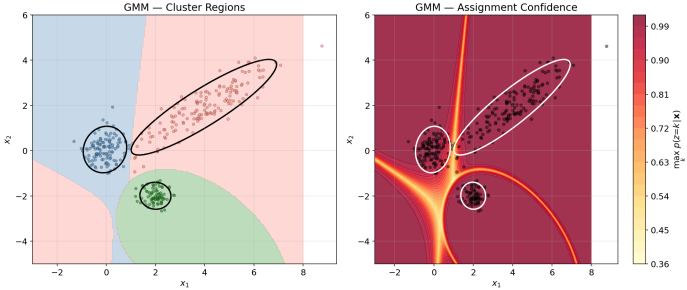
	k-**means**	**GMM**
Assignments	Hard (0 or 1)	**Soft** (probabilities)
Shape	Spherical only	Any (via $\mathbf{\Sigma}_k$)
Uncertainty	None	Per-point uncertainty
Model selection	Heuristic	Likelihood-based
Generative	No	**Yes** — can sample new data

GMM is the **probabilistic version** of k-means — just as logistic regression is the probabilistic version of a linear classifier.

Soft Assignments

Instead of "point i belongs to cluster 2", a GMM says:

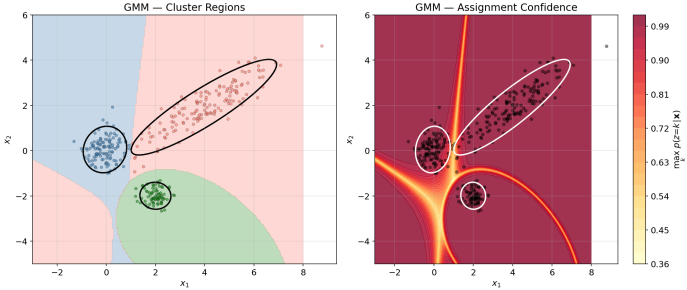
"Point i belongs to cluster 1 with probability 0.1,
cluster 2 with probability 0.85, cluster 3 with probability 0.05."



Soft Assignments

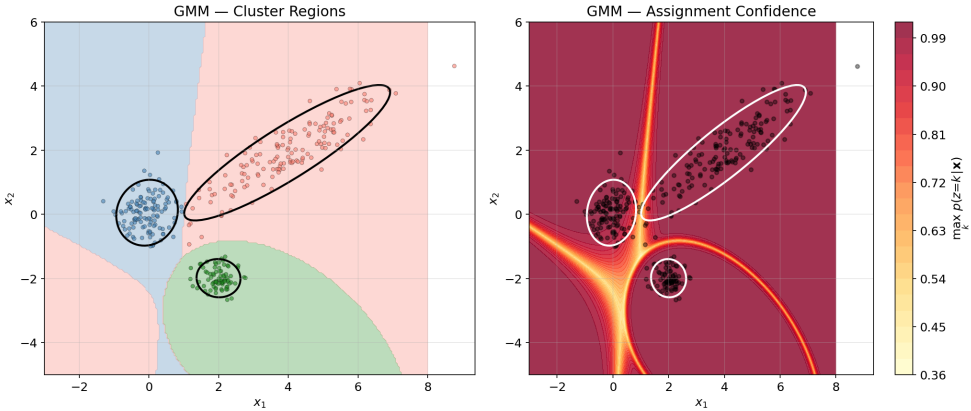
Instead of "point i belongs to cluster 2", a GMM says:

"Point i belongs to cluster 1 with probability 0.1,
cluster 2 with probability 0.85, cluster 3 with probability 0.05."



Points near cluster boundaries have **uncertain assignments** — the model knows it's not sure.

Visualizing Assignment Uncertainty



Uncertain points are **prone to misclassification**. Removing them from downstream analysis **improves clustering accuracy**.

Why the Gaussian? From Euclidean to Mahalanobis Distance

The log-density of a multivariate Gaussian is a **generalized distance**:

$$\log \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = -\frac{1}{2} \underbrace{(\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)}_{\text{Mahalanobis distance}^2} - \frac{1}{2} \log |\boldsymbol{\Sigma}_k| + \text{const}$$

Why the Gaussian? From Euclidean to Mahalanobis Distance

The log-density of a multivariate Gaussian is a **generalized distance**:

$$\log \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = -\tfrac{1}{2} \underbrace{(\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)}_{\text{Mahalanobis distance}^2} - \tfrac{1}{2} \log |\boldsymbol{\Sigma}_k| + \text{const}$$

Special case — when $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$ (k-means assumption):

$$(\mathbf{x} - \boldsymbol{\mu}_k)^\top (\sigma^2 \mathbf{I})^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) = \frac{1}{\sigma^2} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2 \qquad \leftarrow \text{ scaled } \textbf{Euclidean distance}$$

Why the Gaussian? From Euclidean to Mahalanobis Distance

The log-density of a multivariate Gaussian is a **generalized distance**:

$$\log \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = -\tfrac{1}{2} \underbrace{(\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)}_{\text{Mahalanobis distance}^2} - \tfrac{1}{2} \log |\boldsymbol{\Sigma}_k| + \text{const}$$

Special case — when $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$ (k-means assumption):

$$(\mathbf{x} - \boldsymbol{\mu}_k)^\top (\sigma^2 \mathbf{I})^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) = \frac{1}{\sigma^2} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2 \qquad \leftarrow \text{ scaled } \textbf{Euclidean distance}$$

General case — with a full $\boldsymbol{\Sigma}_k$:

- The **Mahalanobis distance** measures distance along the cluster's own axes.
- It accounts for **elongation** and **rotation** of the cluster.
- Contours of equal probability are **ellipses**, not circles.

Why the Gaussian? From Euclidean to Mahalanobis Distance

The log-density of a multivariate Gaussian is a **generalized distance**:

$$\log \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = -\tfrac{1}{2} \underbrace{(\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)}_{\text{Mahalanobis distance}^2} - \tfrac{1}{2} \log |\boldsymbol{\Sigma}_k| + \text{const}$$

Special case — when $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$ (k-means assumption):

$$(\mathbf{x} - \boldsymbol{\mu}_k)^\top (\sigma^2 \mathbf{I})^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) = \frac{1}{\sigma^2} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2 \qquad \leftarrow \text{ scaled } \textbf{Euclidean distance}$$

General case — with a full $\boldsymbol{\Sigma}_k$:

- The **Mahalanobis distance** measures distance along the cluster's own axes.
- It accounts for **elongation** and **rotation** of the cluster.
- Contours of equal probability are **ellipses**, not circles.

The covariance in the exponent is not mysterious — it is the same notion of "distance to center", but generalized to non-spherical clusters.

The Likelihood of a GMM

The marginal likelihood of a single data point (summing over all clusters):

$$p(\mathbf{x}_i \mid \theta) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

The Likelihood of a GMM

The marginal likelihood of a single data point (summing over all clusters):

$$p(\mathbf{x}_i \mid \theta) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

For all data (assuming independence):

$$p(\mathcal{D} \mid \theta) = \prod_{i=1}^{N} \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

The Likelihood of a GMM

The marginal likelihood of a single data point (summing over all clusters):

$$p(\mathbf{x}_i \mid \theta) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

For all data (assuming independence):

$$p(\mathcal{D} \mid \theta) = \prod_{i=1}^{N} \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

We can maximize this to learn $\boldsymbol{\pi}, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k$.

The likelihood also lets us **compare models with different** K.

From Soft to Hard: Watching σ^2 Shrink

With equal weights and $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$, the responsibility simplifies to:

$$z_{ik} = \frac{\exp\left(-\frac{\|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2}{2\sigma^2}\right)}{\sum_{j=1}^{K} \exp\left(-\frac{\|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2}{2\sigma^2}\right)} \qquad \longleftarrow \textbf{softmax} \text{ with temperature } \sigma^2$$

From Soft to Hard: Watching σ^2 Shrink

With equal weights and $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$, the responsibility simplifies to:

$$z_{ik} = \frac{\exp\left(-\frac{\|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2}{2\sigma^2}\right)}{\sum_{j=1}^{K} \exp\left(-\frac{\|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2}{2\sigma^2}\right)} \quad \longleftarrow \quad \textbf{softmax} \text{ with temperature } \sigma^2$$

$\sigma^2 = 2.0$	$\sigma^2 = 0.5$	$\sigma^2 = 0.1$	$\sigma^2 \to 0$
Very blurry	Boundaries visible	Nearly hard	$=$ k-**means**
$z_{ik} \approx \frac{1}{K}$ everywhere	soft transitions	$z_{ik} \approx \{0, 1\}$	$z_{ik} \in \{0, 1\}$ exactly

From Soft to Hard: Watching σ^2 Shrink

With equal weights and $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$, the responsibility simplifies to:

$$z_{ik} = \frac{\exp\left(-\frac{\|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2}{2\sigma^2}\right)}{\sum_{j=1}^{K} \exp\left(-\frac{\|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2}{2\sigma^2}\right)} \quad \longleftarrow \quad \textbf{softmax} \text{ with temperature } \sigma^2$$

$\sigma^2 = 2.0$	$\sigma^2 = 0.5$	$\sigma^2 = 0.1$	$\sigma^2 \to 0$
Very blurry	Boundaries visible	Nearly hard	$=$ k-**means**
$z_{ik} \approx \frac{1}{K}$ everywhere	soft transitions	$z_{ik} \approx \{0, 1\}$	$z_{ik} \in \{0, 1\}$ exactly

As $\sigma^2 \to 0$: **softmax** $\to$ **argmax**, uncertain regions shrink to sharp boundaries.

This is the same "temperature" idea as in neural network softmax outputs. Low temperature = more confident = harder decisions.

Parameter Updates: k-means vs. GMM (EM)

k-means

E-step: assign to nearest center

$$z_i^* = \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

M-step: recompute means

$$\boldsymbol{\mu}_k = \frac{\sum_{i:\, z_i^* = k} \mathbf{x}_i}{\sum_{i:\, z_i^* = k} 1}$$

(unweighted average of assigned points)

GMM — EM algorithm

E-step: compute responsibilities

$$z_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

M-step: weighted averages

$$\boldsymbol{\mu}_k = \frac{\sum_i z_{ik}\, \mathbf{x}_i}{\sum_i z_{ik}}, \quad \pi_k = \frac{\sum_i z_{ik}}{N}$$

$$\boldsymbol{\Sigma}_k = \frac{\sum_i z_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^\top}{\sum_i z_{ik}}$$

Parameter Updates: k-means vs. GMM (EM)

k-means

E-step: assign to nearest center

$$z_i^* = \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

M-step: recompute means

$$\boldsymbol{\mu}_k = \frac{\sum_{i:\, z_i^* = k} \mathbf{x}_i}{\sum_{i:\, z_i^* = k} 1}$$

(unweighted average of assigned points)

GMM — EM algorithm

E-step: compute responsibilities

$$z_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

M-step: weighted averages

$$\boldsymbol{\mu}_k = \frac{\sum_i z_{ik}\, \mathbf{x}_i}{\sum_i z_{ik}}, \quad \pi_k = \frac{\sum_i z_{ik}}{N}$$

$$\boldsymbol{\Sigma}_k = \frac{\sum_i z_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^\top}{\sum_i z_{ik}}$$

Key insight

The *only* difference: k-means uses **binary** weights (0 or 1), EM uses **soft** weights ($z_{ik} \in [0, 1]$). When $z_{ik} \to \{0, 1\}$, EM $=$ k-means.

In Pyro we use SVI (gradient descent) instead of EM, but the optimizer finds the same solution.

Let's See It in Code

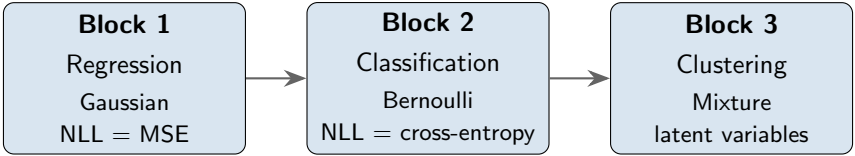
Time to open the **Pyro notebook**!

What we will implement:

1. Gaussian Mixture Model in Pyro.
2. Train via MLE.
3. Visualize **soft cluster assignments**.
4. Compare with k-means.

Same Pyro primitives: `pyro.sample`, `pyro.param`, `pyro.plate`.
New concept: latent variables (z_i).

Today's Journey



In all three blocks:

- Same recipe: **define model** → **infer parameters** → **predict with uncertainty**.
- Same tool: **Pyro** (`pyro.param`, `pyro.sample`, `pyro.plate`, `SVI`).
- Simple model → neural network extension (Blocks 1 & 2).

What We Have Not Modeled Yet

Today we used **Maximum Likelihood Estimation** throughout.

MLE gives us **point estimates** of the parameters: a single $\hat{w}$, $\hat{b}$, $\hat{\sigma}$.

What We Have Not Modeled Yet

Today we used **Maximum Likelihood Estimation** throughout.

MLE gives us **point estimates** of the parameters: a single $\hat{w}$, $\hat{b}$, $\hat{\sigma}$.

But what if we have **little data**?

- Our estimated parameters could be far from the truth.
- Our predictions would be **confident but wrong**.
- MLE cannot tell us: "I'm not sure about the parameters themselves."

What We Have Not Modeled Yet

Today we used **Maximum Likelihood Estimation** throughout.

MLE gives us **point estimates** of the parameters: a single $\hat{w}$, $\hat{b}$, $\hat{\sigma}$.

But what if we have **little data**?

- Our estimated parameters could be far from the truth.
- Our predictions would be **confident but wrong**.
- MLE cannot tell us: "I'm not sure about the parameters themselves."

This is **epistemic uncertainty** — uncertainty about the model itself, not just about the data.

Tomorrow: Bayesian inference puts distributions over parameters, capturing this uncertainty.

Bridging to Day 2

	Day 1 (today)	**Day 2 (tomorrow)**
Inference	MLE	**Bayesian inference**
Parameters	Point estimates $\hat{\theta}$	**Distributions** $p(\theta \mid \mathcal{D})$
Uncertainty	Aleatoric only	Aleatoric + **epistemic**
Pyro guide	Empty	Variational approximation
Models	Regression, classification, GMM	+ **VAEs, deep generative models**

Bridging to Day 2

	Day 1 (today)	**Day 2 (tomorrow)**
Inference	MLE	**Bayesian inference**
Parameters	Point estimates $\hat{\theta}$	**Distributions** $p(\theta \mid \mathcal{D})$
Uncertainty	Aleatoric only	Aleatoric + **epistemic**
Pyro guide	Empty	Variational approximation
Models	Regression, classification, GMM	+ **VAEs, deep generative models**

The good news: **the Pyro model stays the same.**
We only change the guide and the inference procedure.

This is the decoupling of modeling and inference in action.

Thank you!

Questions?

All notebooks and slides are available in the course repository.